Automated E-Learning Data Pipeline

Data Flow & System Behavior

DFDs • Sequence Diagrams • Activity Diagram

Repository: Mahmoud-E-28/Automated_E-Learning_Data_Pipeline
Document Version 1.0

Table of Contents

1. Introduction

This document describes how data moves through the Automated E-Learning Data Pipeline and how the system behaves during execution. It complements the architectural and dbt-layer documentation by focusing specifically on data flow and process behavior, using three complementary modeling notations:

- **Data Flow Diagrams (DFDs) —** show what data moves between which processes and data stores, at both a context (system boundary) level and a detailed (internal pipeline) level.
- **Sequence Diagrams —** show the time-ordered interactions between components during an actual pipeline run, for both the cloud (Microsoft Fabric) and local (Apache Airflow) implementations.
- **Activity Diagram —** shows the workflow logic of a pipeline run as a sequence of activities and decision points, including the fail-fast behavior triggered by data quality gates.

All diagrams are derived directly from the implemented dbt project (staging, intermediate, snapshot, and marts layers), the Fabric pipeline definition (Pipelining/Pipelining .json), and the Airflow DAG used in the local solution.

2. Data Flow Diagrams (DFD)

Data Flow Diagrams model the pipeline using three standard elements: external entities (sources/sinks of data, drawn as rectangles), processes (transformations, drawn as circles), and data stores (persisted tables/views, drawn as open-ended rectangles). Arrows represent the data flowing between them.

2.1 Context-Level DFD (Level 0)

The context diagram treats the entire pipeline as a single process and shows its boundary with the outside world: the three source platforms that supply raw data, the people/systems that trigger and consume it, and the two downstream consumers of its output.

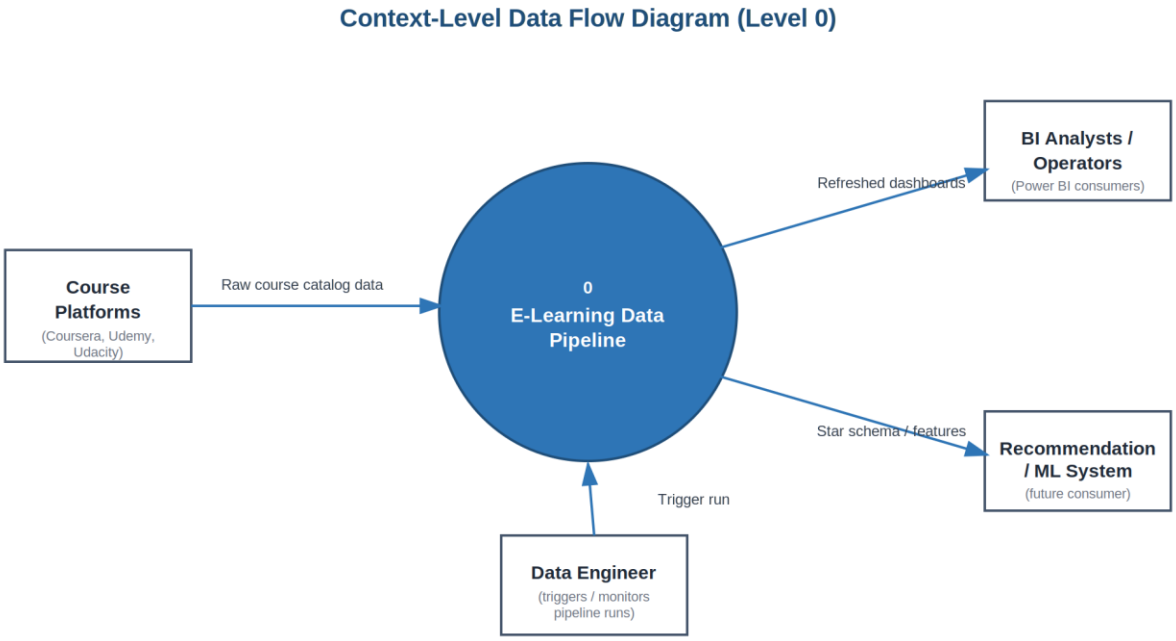

Figure 1 — Context-Level Data Flow Diagram (Level 0)

2.2 Detailed DFD (Level 1)

The Level 1 diagram decomposes the single context process into the pipeline's seven internal processes, mapped directly to the dbt project layers and the Fabric pipeline activities, along with the five data stores each process reads from or writes to.

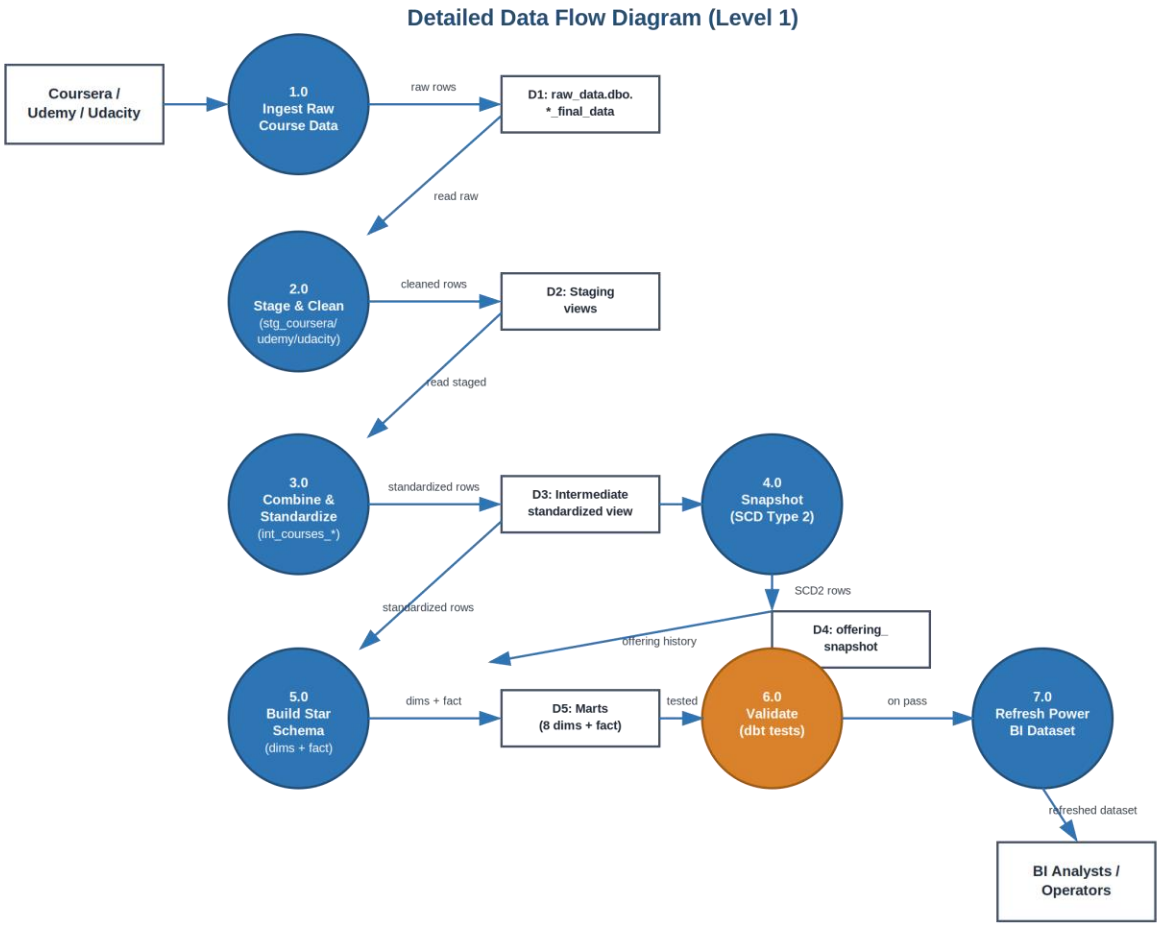

Figure 2 — Detailed Data Flow Diagram (Level 1)

2.3 Data Store Reference

The data stores referenced in the Level 1 diagram correspond to the following physical or logical objects in the dbt project:

ID	Data Store	Implementation
D1	raw_data.dbo.*_final_data	Raw source tables loaded by the Fabric Notebook / pre-populated DuckDB tables (local)
D2	Staging views	stg_coursera, stg_udemy, stg_udacity (materialized as views)
D3	Intermediate standardized view	int_courses_combined → int_courses_standardized (views)
D4	offering_snapshot	dbt snapshot table implementing SCD Type 2
D5	Marts	8 dimension tables + fact_offerings (materialized as physical tables)

3. Sequence Diagrams

Sequence diagrams capture the time-ordered request/response interactions between system components during a single pipeline execution. Solid arrows represent requests or invocations; dashed arrows represent responses or completion signals. Both implementations of the pipeline are documented below.

3.1 Cloud Solution — End-to-End Pipeline Run

This sequence covers a full run of the Fabric Data Pipeline ('Pipelining'), from trigger through to a refreshed Power BI dataset. Each of the three activities (Notebook1, dbt job1, Semantic model refresh1) is gated by the 'Succeeded' dependency condition of the activity before it, so the diagram below reflects the strict, sequential, fail-fast nature of the orchestration.

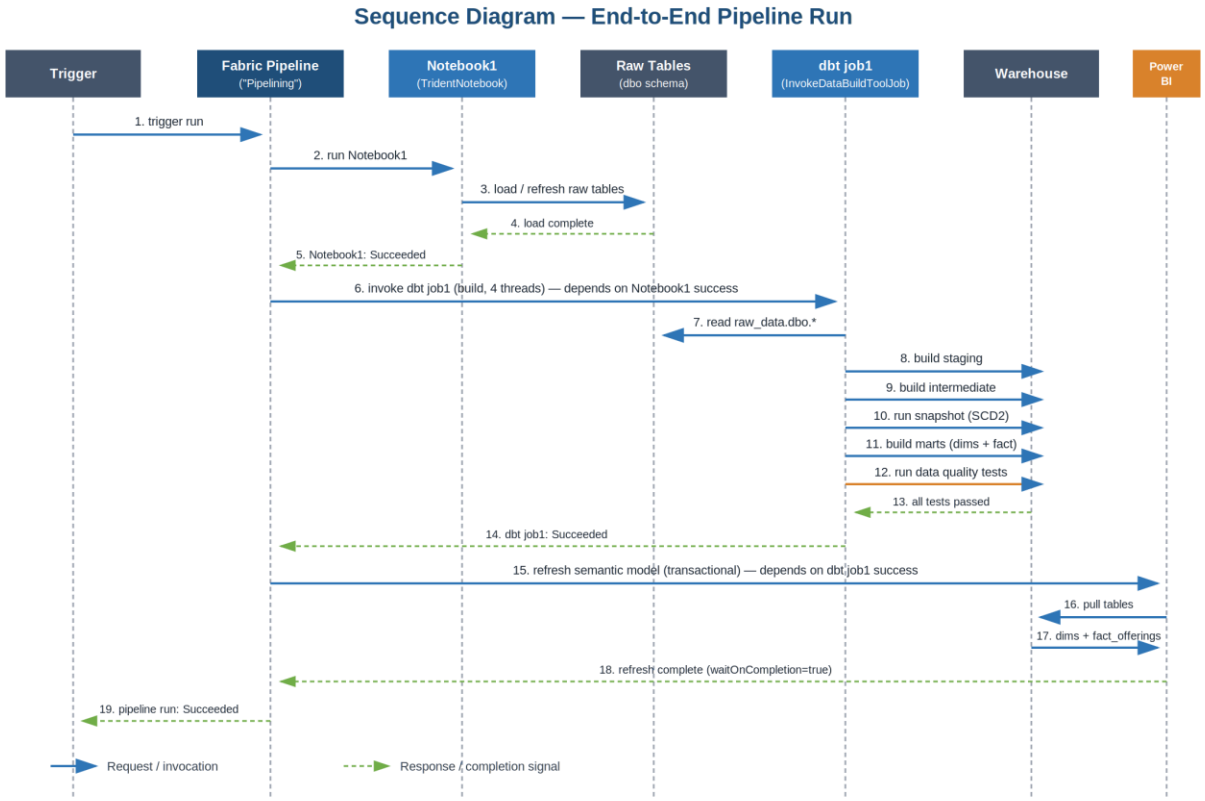

Figure 3 — Sequence Diagram: End-to-End Pipeline Run (Cloud / Microsoft Fabric)

3.2 Local Solution — Airflow DAG Execution

This sequence covers the equivalent run in the local solution, where the course_data_pipeline DAG executes two Airflow tasks against a DuckDB warehouse: dbt_deps (dependency installation) followed by dbt_build (the full model build and test run), avoiding the dependency-cycle issue that separate snapshot/run/test commands would otherwise create.

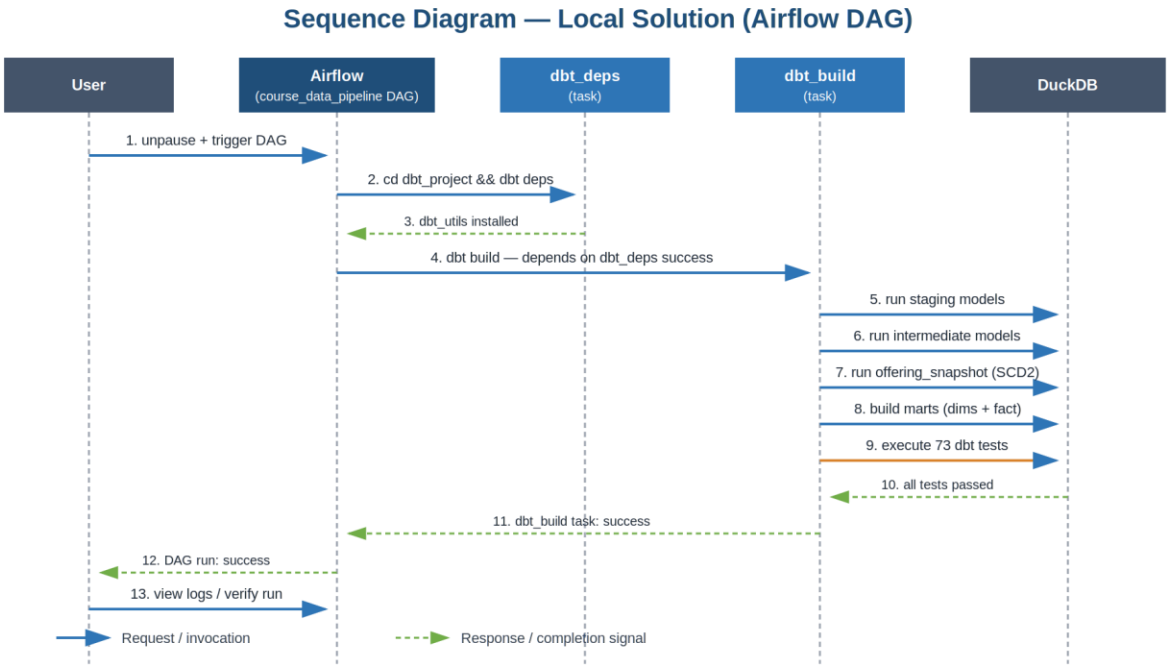

Figure 4 — Sequence Diagram: Local Solution (Airflow DAG)

4. Activity Diagram

4.1 Pipeline Workflow with Decision Points

The activity diagram below visualizes the pipeline as a workflow of activities connected by control flow, including the three decision points that determine whether execution continues or terminates in failure. This reflects the fail-fast design shared by both the cloud and local implementations: a failure at any gate stops the workflow before it can propagate bad data downstream.

- **Gate 1 — Load succeeded?** If raw data ingestion fails, the dbt job is never invoked.
- **Gate 2 — All tests pass?** If any dbt test fails (uniqueness, not-null, accepted values, or referential integrity), the Power BI / reporting layer is not refreshed with incomplete or invalid data.
- **Gate 3 — Refresh succeeded?** The pipeline only reports overall success once the semantic model refresh is confirmed complete (transactional, wait-on-completion).

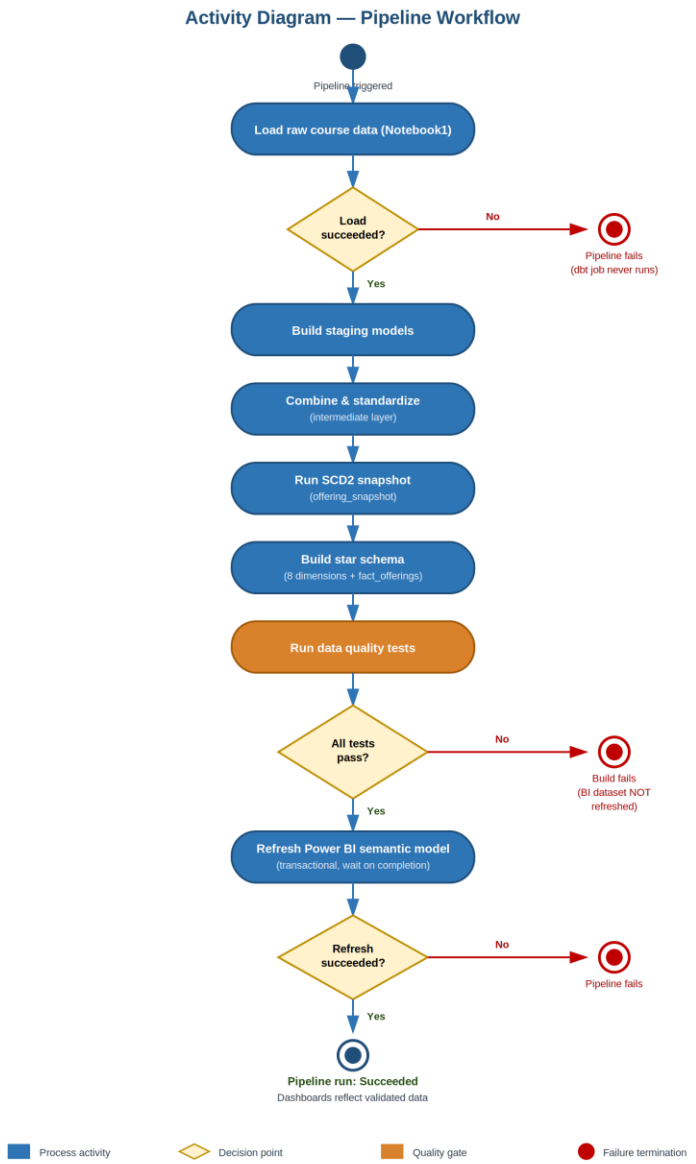

Figure 5 — Activity Diagram: Pipeline Workflow

End of document.